CHAPTER 2-NEED OF PROGRAMMING AND INTRODUCTION TO OBJECT ORIENTED PROGRAMMING APPROACH

2.1Objective

- Introduction to problem solving approach
- Why problem solving
- Step by step approach to problem solving
- Key elements in problem solving
- To introduce programming as a means to automate a solution to any problem
- To understand the term 'Object-Oriented Programming'
- To know about Classes and Objects in the context of Object-oriented programming
- To introduce programming as a means to automate a solution to any problem
- To understand the term 'Object-Oriented Programming'
- To know about Classes and Objects in the context of Object-oriented programming

2.2Content

2.2.1Introduction to problem solving approach

Most people spend most of their time, at work or at home, solving problems. Most problems we face are small, some are large and complex, but they all need to be solved in a satisfactory way.

let's take a few moments to understand what we mean by a problem and problem solving.

A problem is a difficulty or challenge, or any situation that needs a solution

Driving a car to office, Making a cup of tea, Solving a crossword puzzle . all these are examples of problems that we face every day.

2.2.2 **Why Problem Solving**

Problem Solving is actually a skill,and there are no universal methods to be followed to solve a problem. Basically there could be multiple solutions to a single problem and one must try all the possible solutions one by one until we come across with the correct and feasible solution.

2.2.3 Step by step approach to problem solving

Effective problem solving involves number of steps or stages. They are

i) Problem Identification

This stage involves detecting and recognizing that there is a problem; identifying the nature of the problem; defining the problem. This phase also involves , structuring the problem. Structuring the problem is all about gaining more information about the problem and increasing understanding. By spending some time defining the problem will helps to understand it more clearly and it will leads you to the next phase.

ii) Identifying possible solutions and Making Decisions

Once you complete the information gathering in the first phase, the next step is to identify all possible solutions. Once the requirements have been finalized, we design a solution(s) to the issue. We might note the steps on a piece of paper or we might take a print out of it. Referring to these artifacts makes our work simpler and gives us a clear idea of how to proceed with a task .Hence design becomes a very important task as far as any system development is concerned.

iii) Implementation

Implementation means acting on the chosen solution .This phase deals with the actual process of implementation of our design. In terms of programming, the code would follow the design and then write code to translate the human-understandable design to a machine understandable language. During implementation more problems may arise especially if identification of the original problem was not carried out fully.

iv) Test the outcome – Testing

The outcome of our implementation needs to be tested before it is actually deployed. Similarly, all code needs to be thoroughly tested to make sure that bugs are not present in it. If any bugs are identified, they need to be removed and the system needs to be checked again for consistency. Hence testing is a very important phase of problem solving.

v) Deployment and Maintenance

The developed and tested outcome is finally given to the customer. The deployment phase makes sure that the software that has been developed works fine in the customer's environment as per the customer's requirements. Maintenance phase deals with the periodic updates that might be required for the application.

2.2.3.1 Example

Let us take an example of making a cup of tea. This is a minor problem but a problem nevertheless. As a human being , we normally follow the established process of making tea. Let us assume that we need to automate the system of making tea and we are building a tea-vending machine. In the beginning, the tea-vending machine does not know what it needs to do and what will be the outcome of its actions. Hence we program the machine to understand our requirements and work accordingly. Assume that we are going to instruct the machine to make tea. We need to think of the following things

1. What should be provided as the ingredients
2. What should be done with the ingredients
3. What will be the outcome

Ingredients of tea will normally be milk,water, sugar and tea-leaves. Hence we instruct the machine to take milk, sugar and tea-leaves as the input.

Now once we have the inputs, we need to take the following steps

1. Boil water
2. Add tea-leaves
3. Add sugar
4. Add milk

We will instruct the machine to perform these steps sequentially. The machine understands this and using the inputs that were provided to it, will solve the problem.

Finally the outcome of these steps will be tea and in the machines terms, this is called as the output.

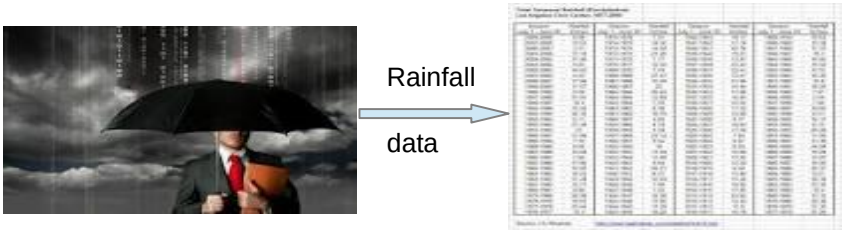
2.2.4 Key elements required for problem solving

2.2.4.1 Good Analytical Skills

Analytical skills are the ability to imagine, collect information, arrange it, analyse, solve complex problems, and make decisions. The ability to organise data and draw proper correlations and then interpreting these trends in terms that are meaningful to others is called Analytical skill.

Let us look a example to understand the good analytical skills

i) Rainfall Data for past thirty years at random are supplied

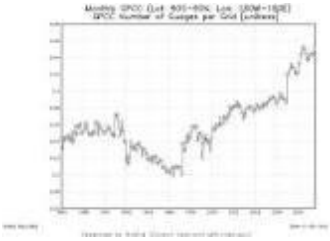


ii) Organize and arrange the data

By feeding it to excel sheet or any logical approach

	A	B	C	D	E	F
1	Monday	Tuesday	Wednesday	Thursday	Friday	
2	264.485	512.38	415.502356	982.9619	599.257	
3	771.484	654.117	774.812805	220.5584	223.456	
4	442.683	183.212	382.736928	238.9387	85.2454	
5	289.324	384.166	759.005851	367.8189	142.585	
6	454.549	191.218	347.149848	633.6318	476.167	
7	657.484	636.238	83.1531866	880.9597	42.2667	
8	571.52	685.743	186.417108	394.3437	639.993	
9	21.7125	80.2912	21.8358466	1.096106	12.7634	
10	74.4262	44.7458	46.6517789	9.593594	69.0575	
11						
12						
13						
14						

Draw graphs to demonstrate the data



iii) **Output**

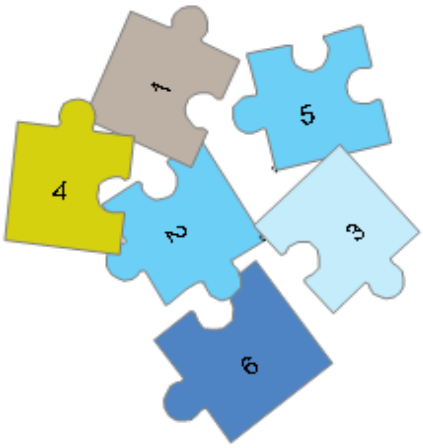
make reasonable predictions about the extent of rain next year.

This is an example of using analytical skills.

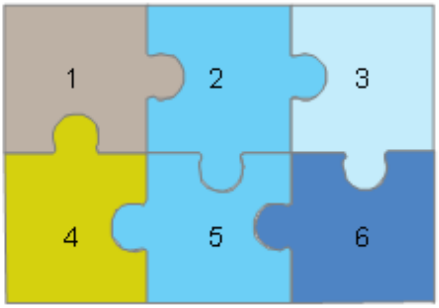
2.2.4.2 Declarative and Imperative Knowledge

i) **Declarative knowledge** is factual knowledge or the knowledge of the solution. For example knowing that "A pen is used for writing" is a declarative knowledge. Declarative knowledge is knowledge or the possession of information that is either true or false. It describes objects and events by specifying the properties which characterize them; it does not pay attention to the actions needed to obtain a result

Take an example of a jigsaw puzzle



We know that the solution of this problem is



so this knowledge of the outcome is know as declarative knowledge.

ii) Imperative Knowledge

Continuing with the above example, although we know the result, we also need to know how to reach the result. This 'how-to' knowledge is called as imperative knowledge. Let us consider the steps

1.Arrange the pieces according to their number
2.Turn the pieces fitting to nearby pieces.
3.Join the pieces

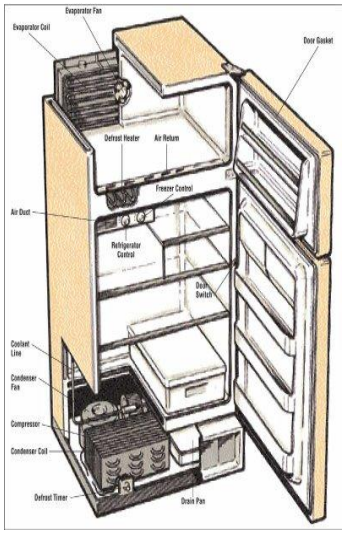
As we can see, if this order is followed, the puzzle would be solved. Hence the knowledge of the steps to reach the solution is called as Imperative knowledge.

2.2.4.3 Abstraction

A representation or model that includes the important, essential or distinguishing aspects of something while suppressing or ignoring less important, immaterial or diversionary details is called as Abstraction. Hence abstraction is a way of managing problem complexity by focusing on the essential details and ignoring lower level details.

An example can be taken of a refrigerator

As a user, we just need to know that a fridge can be used to preseve food materials by freezing them. Hence the user just needs to open the fridge and put the items inside.



User does not need to know the internal functioning of the fridge, hence the wiring,electronics and other details are hidden behind the body of the fridge. This process of hiding the irrelevant data is called as Abstraction.

2.2.4.3.1 Abstraction types

i) Functional Abstraction

Let see the above refrigerator example, all refrigerator contains a **freezer box** which can preserves the food material below zero degree Celsius temperature. As a user we need to get the food material to be preserved but we don't need how it will be get preserved , this way of hiding irrelevant details or process and getting the functionality of preservation to be done is an example of functional abstraction.

ii) Data Abstraction

Let see the above refrigerator example,the inner temperature of the refrigerator is an essential data which is sensed by the sensor and refrigerator monitors and controls the inner temperature. Here inner temperature is the internal data which is used by the refrigerator to maintain the temperature of the fridge, which is not controlled or altered by external mechanism and is used by the functionalities of the refrigerator is an example of data abstraction.

2.2.4.4 Knowledge in programming language

```
def add5(x):
   return x+5

def dotwrite(ast):
   nodename = getNodename()
   label=symbol.sym_name.get(int(ast[0]),ast[0])
   print '    %s [label="%s' % (nodename, label),
   if isinstance(ast[1], str):
      if ast[1].strip():
         print '= %s"];' % ast[1]
      else:
         print '"]'
   else:
      print '"];'
      children = []
      for  in n, childenumerate(ast[1:]):
         children.append(dotwrite(child))
      print ,'    %s -> {' % nodename
      for  in :namechildren
         print '%s' % name,
```

A programming language is a computer language programmers use to develop applications, scripts, or other set of instructions for a computer to execute.

Programming languages can be used to create programs that control the behaviour of a machine and/or to express algorithms precisely.

Thousands of different programming languages have been created, mainly in the computer field, with many more being created every year.

2.2.4.4.1**Elements of a programming language are -**

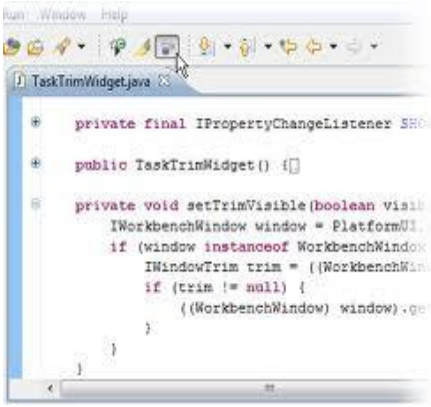
i)Syntax: structural elements of the language. Programs must be syntactically correct.
Eg: English as a language has got a certain syntax. We have predefined syllables and words like 'a' or 'cup' or 'welcome'. These words convey a meaning to the language.

ii) **Grammar**: defines how syntactical elements need to be combined to form programs
Eg: Continuing the same example, English language has got a certain grammar. The grammar has to be followed to construct a proper sentence. 'I is a boy' is grammatically wrong although syntactically correct.'I am a boy' is syntactically and grammatically correct.

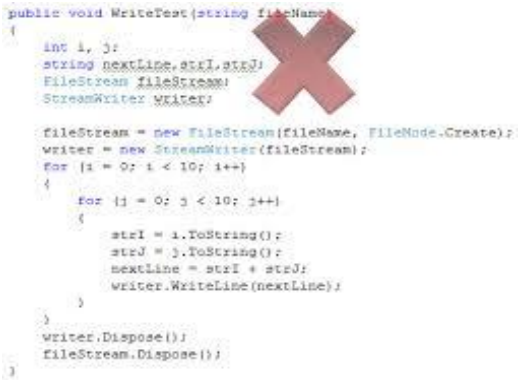
iii) **Semantics**: defines meaning of the code
Eg: Continuing the same example, English language has got a certain semantics. 'An elephant is white and has got two legs' is syntactically , grammatically correct. But it doesn't make any sense. Hence it is semantically wrong.

2.2.4.5 Good programming practices

i) Good vs Bad programming



proper variables, aligned properly



unused variables, not aligned

Superior coding techniques and programming practices are hallmarks of a professional programmer. The coding techniques are primarily those that improve the readability and maintainability of code,whereas the programming practices are mostly performance enhancements.

ii)

The readability of source code has a direct impact on how well a developer comprehends a software system.

iii)**Maintainability**

Code maintainability refers to how easily that software system can be changed to add new features, modify existing features, fix bugs, or improve performance

Although readability and maintainability are the result of many factors, one particular facet of software development upon which all developers have an influence is coding technique. The easiest method to ensure that a team of developers will yield quality code is to establish a **coding standard,** which is then enforced at routine code reviews.

2.2.5 **Evolution of Programming Approaches**

As we have seen in the previous course, for automating the solution of a problem, we need to do programming. Originally when the programming evolved, the adopted approach was small with complex syntax and machine code. With time, as requirements increased, the logic became much more complex with increase in the program size.

The following are the different programming approaches which evolved were:

- Unstructured Programming
- Structured Programming
- Procedural Programming
- Modular Programming
- Object Oriented Programming

The initial evolution methodologies had the following disadvantages

- Large volume of code and logic
- Lack of information hiding
- Poor readability
- Non-maintainable
- Reduced reusability

2.2.6 Elements of Programming

For programming to be done, we need the following

- ✓ What is the information that is to be processed in the problem **- Data**
- ✓ What are the types or ways to process the information **- Logic**
- ✓ How will the processing happen using the logic – **Methodology**

2.2.7 Structured Programming Approach

Structured approach to programming, implements a system as a collection of functions that accepts data, processes it and returns an output. A function sometimes calls another function, when it wants to make use of the functionality defined in the other function. The flow of program execution is based on the structure of the program written. There is more dependency between the variables and the program like a chain.

Example: C language

The way of structured programming doesn't relate to real world problem scenarios. The current problem scenarios doesn't call for functions that operate on data, rather the scenarios have data and their associated behaviour. In such cases, the structured way of programming becomes very huge and cumbersome. Information has to be manipulated across all functions and an association of varied information in a problem scenario is not possible.

2.2.8 Object Oriented Programming Approach

We write programs that make up software to solve a problem in real world. In real world problem domains we do not find any functions that work on data. We find objects that are described by some state (data) and having some behaviour (functions or routines).

Object Oriented (OO) Approach to programming is about implementing a system as collection of objects that have state and behaviours, interacting with each other to achieve expected functionalities.

Example: C++ & Java

In OO programming, the program basic entity is object. An object is any real world entity that has a well-defined structure and behaviour.

Forget programming for a while. Think about the Real World and the things that are in it. What things are objects? What things are not objects?

Look around you. You may try to list five objects and list five non-objects.

	Objects	**Non-Objects**
1		
2.		
3.		
4.		
5.		

	Objects	**Non-Objects**
1.	Pen	The upper 37% of pen
2.	Computer Keyboard	The air above the keyboard
3.	Shoe	Colour of the shoe
4.	Mouse	Sound of the mouse click
5.	Car	Speed of the car

It is easier to list things that are objects than to list things that are not objects. Just to talk about something seems to make it an object, somehow. When we observe, the humans view the world in object-oriented terms. The human brain wants to think about objects, and our thoughts and memories are organized into objects and their relationships. Perhaps non-human brains work differently.
One of the ideas of object-oriented software is to organize software in a way that matches the thinking style of our object-oriented brains.

Look at my list of objects and your own list and try to describe what all objects have in common. What makes an object?

- An object is made of tangible material (the pen is made of plastic, metal, ink).
- An object holds together as a single whole (the whole pen, not a fog).
- An object has properties (the color of the pen, where it is, how thick it writes.).
- An object can do things and can have things done to it.

The first item in this list is too restrictive. For example, you can think of your bank account as an object, but it is not made of material. (Although you and the bank may use paper and other material in

keeping track of your account, your account exists independently of this material.) Although it is not material, your account has properties (a balance, an interest rate, an owner) and you can do things to it (deposit money, cancel it) and it can do things (charge for transactions, accumulate interest).

The last three items on the list seem clear enough. In fact, they have names:

i. An object has identity (each object is a distinct individual).

ii. An object has state (it has various properties, which might change).

iii. An object has behavior (it can do things and can have things done to it).

iv. This is a somewhat ordinary description of what an object is like. (This list comes from the book Object-oriented Analysis and Design, by Grady Booch, Addison-Wesley, 1994.) Do not be surprised if other notes and books have a different list. When you start writing object-oriented software you will find that this list will help you decide what your objects should be.

Every object has its own characteristics. Every object has 5 sets of characteristics Viz. State, Behaviour, Responsibility, Communication and Identity.

2.2.9 Characteristics of an object

The five characteristics of an Object are explained below:

States of an Object: The states of an object are known by the various attributes that describe the Object. Example: If Human is an Object, colour of Hair, number of Teeth, skin tone, etc., describe the object. These form the states of the Object, Human

More Examples:

i. Make of Tyres, Colour, Engine Type, Mileage of a Car Object.
ii. Account Number, Account Holder Name, Deposit Amount, Transaction Details in a Bank Account Object.
iii. Employee ID, Employee Name, Date of Birth, Date of Joining, Number of Dependants in an Employee Object.

Behaviours of an Object: The behaviours of an Object is known by its outcome or reaction whenever there is a change in the states of it. The changes can also be infused by operations which are performed on the states of the object.

Examples:

- ✓ Add, Update, Remove dependant information of an Employee object
- ✓ Calculate performance of the Car based on engine type and mileage
- ✓ Print transaction details in a Bank Account

Responsibilities of an Object: The responsibilities of an Object are of two types namely Knowing Responsibility and Doing Responsibility

- Knowing Responsibility – Knowledge of the states of the Object
- Doing Responsibility – Operations performed on the states of the object

Examples:
The knowledge of the states like knowing the dependant details, retrieving date of birth of an employee are knowing responsibility.
All operational logic performed on the states of the object through its behaviours constitutes the Doing responsibility. Example: Calculate performance of the car

Communication between Objects: In a typical problem scenario, several objects communicate with each other to perform operations, send messages and return data.
Example: Employee object has a salary account in a bank. So both objects communicate with each other to perform all bank related transactions of the employee.

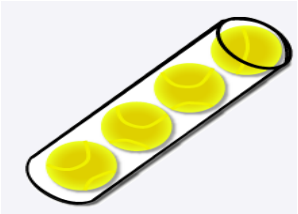
Identity of Objects: Identity of objects as explained earlier relates to the distinctness of the object. We identify the objects in a problem scenario based on all nouns of the project. The behaviours are identified using the verbs operated upon by the nouns

Example: Employee updating Bank Account information to Corporate Portal.

Distinct Objects: Employee, Bank Account, Portal
Behaviours: Updating the related information among the objects.

To enable understand the characteristics of an object, let us consider the following example

Consider a tube of four yellow tennis balls.



- Is the tube of tennis balls an object?
- Is each tennis ball an object?
- Could the top two balls be considered a single object?
- Is the color of the balls an object?
- Is your understanding of tennis balls an object?

Answers:

Is the tube of tennis balls an object?

Yes. It has identity (my tube of balls is different than yours), it has state (opened, unopened, brand name, location), and behavior (although not much).

Is each tennis ball an object?

Yes. It is OK for objects to be part of other objects. Although each ball has nearly the same state and behavior as the others, each has its own identity.

Could the top two balls be considered a single object?

Not ordinarily. Each has its own identity independent of the other. If they were joined together with a stick you might consider them as one object.

Is the color of the balls an object?

No. It is a property of each ball.

Is your understanding of tennis balls an object?

Probably not, although it is unclear what it is. Perhaps it is a property of the object called "your brain."

Software Objects:

Many programs are written to do things that are concerned with the real world. It is convenient to have "software objects" that are similar to "real world objects". This makes the program and what it does easier to think about. Software objects have identity, state, and behavior just as do real world objects. Of course, software objects exist entirely within a computer system and don't directly interact with real world objects.

An object stores its state in fields (variables in some programming languages) and exposes its behavior through methods (functions in some programming languages).

What are software objects made out of?

Computer Memory.

Its important to ask yourself :

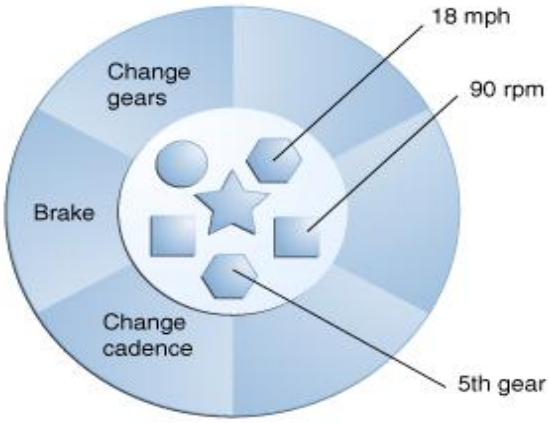
- **What possible states can this object be in?**
- **What possible behavior can this object perform?**

  Think your table lamp may have only two possible states (on and off) and two possible behaviors (turn on, turn off), but your table radio might have additional states (on, off, current volume, current station) and behavior (turn on, turn off, increase volume, decrease volume, seek, scan, and tune).These real-world observations all translate into the world of object-oriented programming.

2.2.10 Classes and Objects

Classes are generic representations of Objects in problem scenarios. They help in generalization of the object as a template. Each object becomes an instance or photocopy of this template.

Consider a car, for example:



Here Car is the template having the generic states and behaviours. The states and behaviour of the template are grouped in the following manner.

```
class Car {
   int speed = 0; //state
   int gear = 1; //state
  // Behaviour
   void changeGear(int newValue) {
       gear = newValue;
   }
  // Behaviour
  void speedUp(int increment) {
       speed = speed + increment;
   }
   void applyBrakes(int decrement) {
       speed = speed - decrement;
```

```
    }

    void printStates() {

        System.out.println("cadence:" +  cadence + " speed:" + speed + " gear:" + gear);

    }

}
```

All various types of Cars which follow the generalization become Objects. Example: Maruti, Volkswagen, Audi, etc.,.

2.2.11 Basic difference between Programming Paradigms

Functional/procedural programming:
Program is a list of instructions to the computer

Object-oriented programming:
Program is composed of a collection objects that communicate with each other

2.3 Reference links

http://en.wikipedia.org/wiki/Problem_solving
http://www.mindtools.com/pages/main/newMN_TMC.ht
http://www.wikihow.com/Improve-Analytical-Skills